

Functions

Krishna Kumar Mohbey¹, Malika Acharya¹

¹ Computer Science, Central University of Rajasthan, India

Abstract

Functions are one of the primary concepts in every programming language. They provide an easy way to package the programming logic and use it as and when required as many times at any place. Thus they help to reduce the redundancy in code and increase the reproducibility. With the increase in the length of code, it is often a good idea to divide the code into separate modules by splitting the code into different functions based on its utility. This is a much-sought practice to organise the lengthy code. They also help in unit testing the code as testing small units in isolation is quite an easy task. This deliberate use of functions thus supports language flexibility and a user-friendly interface. In this chapter the pertinent takeaways would be:

1. Understanding *functions* and *function calls*.
2. Comprehending the concept of *local* and *global* variables.
3. Programming with the recursive functions.

Keywords: Reproducibility, Redundancy, Reusability, Recursion.

INTRODUCTION

The code may be organized, made more understandable, reused, and repurposed with the use of functions, which are a helpful approach to separate the code into more manageable portions. In addition, a function is a piece of code that is put into action when the function itself is invoked. It can take in data in the form of a parameter or argument and then return the result.

Definition

A function is an area of code that comprises a block of statements that carry out a particular operation. When creating a function, there are a few fundamental guidelines to follow:

1. Function blocks start with the **def** keyword. After that, the **function name** and parentheses (**()**) are used.
2. An argument or parameters should be passed inside the parentheses.
3. Any function's code block begins with a colon (**:**) and is indented.

Syntax:

```
def function_name(parameters):  
    Statement 1  
    Statement 2  
    Statement 3  
    -  
    -  
    -  
    Statement n  
    return [expression]
```

There are two types of functions, as described below:

- Predefined functions
- User-defined functions

PREDEFINED FUNCTIONS

It is sometimes referred to as a built-in function because Python has its functionality established in the language. The Python interpreter has several built-in functions that are always available for usage. There are many kinds of built-in functions, and here is an example of some of those built-in functions (Table [1](#)):

Table 1 **Predefined functions.**

<i>abs()</i>	It returns the absolute value of a number
<i>bin()</i>	It returns the binary version of a number
<i>float()</i>	It returns a floating-point number
<i>hex()</i>	It converts a number into a hexadecimal value
<i>int()</i>	It returns an integer number
<i>len()</i>	It returns the length of an object
<i>list()</i>	It returns a list
<i>max()</i>	It returns the largest item in an iterable
<i>min()</i>	It returns the smallest item in an iterable
<i>oct()</i>	It converts a number into an octal
<i>pow()</i>	It returns the value of x to the power of y
<i>print()</i>	It prints to the standard output device
<i>range()</i>	It returns a sequence of numbers, starting from 0 and increments by 1 (by default)
<i>round()</i>	It rounds a numbers

Let's look at an example of each of the built-in functions discussed so far to better grasp how they should be used.

Example:

```
1.  #Return the absolute value of a number
2.  x1 = abs(-5.37)
3.  print(x1)
4.
5.  # Return the binary version of 24
6.  x2 = bin(24)
7.  print(x2)
8.
9.  #Convert the number 7 into a floating-point number
10. x3 = float(7)
11. print(x3)
12.
13. #Convert 321 into hexadecimal value
14. x4 = hex(321)
15. print(x4)
16.
17. #Convert the number 7.2 into an integer
18. x5 = int(7.2)
19. print(x5)
20.
21. #Return the number of items in a list
22. list1 = ["Malika", "Krishna", "William", "Sam"]
23. x6 = len(list1)
24. print(x6)
25.
26. #Return the largest number
27. x8 = max(245, 433)
28. print(x8)
29.
30. #Return the lowest number
31. x9 = min(343, 434)
32. print(x9)
33.
34. #Convert the number 12 into an octal value
35. x10 = oct(32)
36. print(x10)
37.
38. x11 = pow(5, 5) #Return the value of 5 to the power of 5 (same as 5 * 5 * 5 * 5)
39. print(x11)
40.
41. #Print a message onto the screen
42. Print("It is a message")
43.
44. #Create a sequence of numbers from 0 to 7, and print each item in the sequence
45. x12 = range(8)
46. for n in x12:
47.     print(n)
48.
49. #Round a number to only two decimals
50. x13 = round(6.66543, 2)
51. print(x13)
```

Output:

```
5.37
0b11000
7.0
0x141
7
4
433
343
0o40
3125
It is a message
0
1
2
3
4
5
6
7
6.67
```

USER-DEFINED FUNCTIONS

A user-defined function is what's known as a user-defined function when it's a function that's been written for a specific job. In addition, the user is the one who designed or wrote the function themselves. Let's look at an example of a user-defined function so that we can have a better grasp of how this process works.

Example 1:

```
1.  # Declaring a show function
2.  def show():
3.      print ("Inside function")
4.
5.  # Calling function
6.  show()
```

Output:

```
Inside function
```

Example 2:

```
1.  # Python function to check even and odd
2.  def evenOdd(var):
3.      if (var%2 == 0):
4.          print ("Number is Even")
5.      else:
6.          print ("Number is Odd")
7.
8.  # Calling function
9.  evenOdd(10)
10. evenOdd(11)
```

Output:

```
Number is Even
Number is Odd
```

Function Call

We may call a function to perform a task once it has been specified. For calling a function, we type the name of the function and the necessary parameters.

Syntax:

```
function_name (argument_1, argument_2)
```

Example 1:

```
1.  # Function Definition
2.  def my_function():
3.      print ("This function has been created")
4.
5.  my_function() # Function Calling
```

Output:

```
This function has been created
```

Example 2:

```
1.  # Function Definition
2.  def my_name(name):
3.      print (name)
4.
5.  my_name('David') # Function Calling with argument
```

Output:

David

Example 3:

```
1. # Function for the sum of three numbers
2. def sum_three_numbers(num1, num2, num3):
3.     return num1 + num2 + num3
4.
5. # Function calling with three arguments
6. x= sum_three_numbers (1,2,3)
7. print (x)
```

Output:

6

Function Parameters and Arguments

A piece of information can be passed into a function based on the arguments and parameters that the function accepts. The precise values sent into a function are referred to as **arguments**. We may provide several arguments, but a comma must break each up. The phrases “parameter” and “argument” are synonymous and can be used in place of one another. In addition, data goes *via* a function to be processed. There is a distinction, although a subtle one, that can be made between function parameters and arguments. Parameter refers to the variable listed within the parenthesis in the function description. On the other hand, “argument” refers to a value that is supplied to the function when it is called. To assist you in comprehending the significance of function arguments and parameters, let's look at an illustration.

Example:

```
1. # parameters example
2. def Name_function(F_name, L_name): # Function Definition
3.     print (F_name + " " + L_name)
4.
5. # Two arguments value (Krishna, Kumar)
6. Name_function("Krishna", "Kumar") # Function Calling
```

Output:

Krishna Kumar

Default Arguments

In Python, default values for function arguments are allowed. The assignment operator (=) can be used to provide a default value for an argument. Consider the following scenario.

Example:

```
1. def my_name(name, msg="how are you?"): #Function Definition
2.     print("Hello", name + ', ' + msg)
3.
4. my_name("John") #Function Calling
5. my_name("James", "Welcome") #Function Calling
```

Output:

```
Hello John, how are you?
Hello James, Welcome
```

Let's look at some other examples related to function.

Example:

```
1. #Python Arbitrary Arguments
2. def some_name(*names):
3.     print("Hello", names)
4.
5. some_name("Jay", "Joy", "Ram", "Amit")
6.
7. #Arguments passing
8. def fun1(food):
9.     for n in food:
10.        print(n)
11.
12. var = ["apple", "banana", "cherry"]
13. fun1 (fruits)
```

Output:

```
Hello ('Jay', 'Joy', 'Ram', 'Amit')
apple
banana
cherry
```

Variable Scope and Lifetime

The variable cannot be accessed from any of the different parts of the program. There is a possibility that some of the variables will not even be present in the entire program. The variable's declaration determines the variable's existence and accessibility.

Scope of the Variable

It is a part of the program in which a variable is accessible. There are two types of the scope of the variable:

- *Local Scope*
- *Global Scope*

Local Scope

A variable is created in the nested function then the scope exists in that function. Suppose the variable is declared in the innermost function; then, the scope of that variable is only in the innermost function.

Global Scope

A global variable is generated in the software's main body and belongs to the global scope. Global variables of this kind are available in any scope. Furthermore, any function, method, or code easily understands the global variable.

The Lifetime of the Variable

It is the duration for which a variable exists. Let's take an example for understanding the variable scope and lifetime concept.

Example:

```
1.  # local scope
2.  def show():
3.      x = 1234 # Local variable
4.      print (x)
5.  show()
6.
7.  x = 1234 # Global Varibale
8.
9.  def show():
10.     print (x)
11.  show()
12.
13. print (x) # Global scope
```

Output:

```
1234
1234
1234
```

Local and Global Variables

Local variables are declared inside the method or block. Local variables are assessible only within the declared method or block and cannot be accessed outside that method or block.

Example:

```
1.  def my_fun():
2.      x=100    #Local variable
3.      print (x)
4.
5.  my_fun()
```

Output:

```
100
```

Global Variables

This type of variable is defined in the main body of a file. It is available throughout the program. All functions or blocks can easily understand and access a global variable.

Example:

```
1. x=10 #Global
2. def my_fun():
3.     x=5 #Local scope within my_fun()
4.     print (x) #Print local variable
5.
6. my_fun()
7.
8. print (x) #Print Global variable
```

Output:

```
5
10
```

Global Statement

The block in which the local variable is stated is the only place in which its scope may be evaluated. In addition, a variable is considered local when constructed inside a function by the programmer. However, using the global keyword within a function, we can create a variable accessible across the program. This is the only place where it may be used. To better understand the concept, let's look at an example of a global statement that uses the global keyword.

Example:

```
1. def my_fun():
2.     global x
3.     x = 10 #x is global
4.
5. my_fun()
6.
7. print ( x) #print x
```

Output:

```
10
```

Suppose we have a variable with the same name as a global variable in the program. In such a case, a new local variable of that name is created, different from the global variable.

Example:

```
1. x=10
2. def show():
3.     x=5
4.     print ("In Function x is=",x)
5. show()
6. print ("Outside function is=",x)
```

Output:

```
In Function x is= 5
Outside function is= 10
```

Return Statement

It's used to finish the function call's execution and return the result (value or expression). There are a few key points to remember regarding **return** statements:

- Return None is the same as a return statement with no arguments.
- The statements following the return statements are skipped.
- The unique value None is returned if there is no expression in the return declaration.
- You can't use a return statement outside of a function.

Syntax:

```
def fun_name():
    function body
    .
    .
    .
    return [expression]
```

Example:

```
1. # return example
2. def addition(n1, n2 ):
3.     total = n1+ n2
4.     return total.
5.
6. total = addition(1, 2 );
7. print ("Addition is =", total)
```

Output:

Addition is = 3

Lambda Functions

Lambda functions are used when an anonymous function or a function that is declared without a name exists. In most cases, functions are defined using the `def` keyword, whereas anonymous functions are defined using the ***lambda*** keyword. The syntax for lambda functions is described below.

The following are essential points about lambda functions:

1. The number of arguments for a lambda function is infinite, but only one expression exists.
2. We use lambda functions if we need an anonymous function for a short time.
3. The meaning of Lambda does not include a “return” statement; instead, it always includes an expression that is returned.

Syntax:

lambda arguments: expression

Example 1:

```
1. # Add 50 to the argument and return
2. res = lambda a : a + 50
3. print (res(25))
```

Output:

75

Example 2:

```
1. # Sum of three numbers
2. res = lambda n1, n2, n3: n1 + n2 + n3
3. print (res (1, 2, 3))
```

Output:

6

Recursive Functions

When we construct a function and then call that function within the definition of the function, we have created a recursive code. Recursion is the name given to the process in which a function calls itself. Programming using recursion is a very effective method; nevertheless, it is important to keep in mind that the recursion must be stopped at some point. It is a sign that the code functions accurately if the ending condition can be found within the recursion. If the termination condition is absent, it indicates that the recursive code will execute an unlimited number of times.

Syntax:

```
def func_name(): <-  
    |  
    | (recursive call)  
    |  
func_name() ----
```

The followings are the advantages and disadvantages of a recursive function:

Advantages

- Using recursion, a function can be broken down into smaller sub-problems.
- Often creating a sequence is more straightforward than nested iteration.
- The use of recursive functions makes the code appear simple and efficient.

Disadvantages

- Recursive calls use a lot of memory and time, which is why they are costly.
- Debugging recursive functions is difficult.
- The logic behind recursion can be challenging to understand at times.

Example:

```
1.  # Program for Fibonacci series up to k terms
2.  def fib(k):    # Recursive function
3.      if k <= 1:
4.          return k
5.      else:
6.          return(fib(k-1) + fib(k-2))
7.
8.  term=8
9.  if term <= 0:
10.     print ("Invalid input ! Please input a positive value")
11. else:
12.     print ("Fibonacci series:")
13.     for i in range(term):
14.         print (fib(i))
```

Output:

```
0
1
1
2
3
5
8
13
```

Example:

```
1.  # Recursive program to compute the factorial value
2.  def fact(k): # Recursive function
3.      if k == 1:
4.          return k
5.      else:
6.          return k * fact(k-1)
7.
8.  n = 5
9.  # check if the input is valid or not
10. if n < 0:
11.     print ("Invalid input ! Please enter a positive number.")
12. elif n == 0:
13.     print ("Factorial of number 0 is 1")
14. else:
15.     print ("Factorial of number", n, "=", fact(n))
```

Output:

```
Factorial of number 5 = 120
```

Function Redefinition

Python provides a facility for the redefinition of functions because Python is dynamic. Redefinition of function means redefining an already defined function. Let's take an example for understanding the concept of Function redefinition.

Example:

```
1.  # function redefinition example
2.  from time import gmtime, strftime
3.
4.  def show(msg):    # Function definition
5.      print (msg)
6.  show("Start")
7.
8.  def show(msg):    # Function redefinition
9.      print (strftime("%H:%M:%S", gmtime()))
10.     print (msg)
11. show ("Processing.")
```

Output:

```
Start
11:20:20
Processing.
```

CONCLUDING REMARKS

In this chapter, we went through the function and how it is crucial for modular programming to focus on the functions. It is concerned with how it is used to send parameters into a function. We have also gone through the differences between local and global variables, statements, and the scope of each. Therefore, this chapter covers every conceivable variant of the function and the process by which it operates. As we move on to the next topic that Python supports, strings, lists, and the associated features will be covered in the following chapter.